TP5 – Gestion des comptes bancaires : cas de test et critères d'acceptation

Périmètre : classes `BankAccount`, `SavingAccount`, `CheckingAccount`, `Bank`, `Menu`, `App` (+ `Operation`).
Conventions : `FREE_TRANSACTIONS = 3`, `TRANSACTION_FEE = 0,50`. Les numéros de compte commencent à 1 et s'auto-incrémentent.

1. Critères d'acceptation

AC-01 Un **BankAccount** peut être créé avec un solde de 0,00 (constructeur par défaut) ou avec un montant initial.

AC-02 Chaque nouveau compte reçoit un **account_id** unique, auto-incrémenté de 1.

AC-03 **deposit(amount)** augmente le solde de *amount* ; un montant inférieur ou égal à 0 est rejeté.

AC-04 **withdraw(amount)** diminue le solde de *amount* ; un montant inférieur ou égal à 0 ou supérieur au solde est rejeté et le solde reste inchangé.

AC-05 **transfer(other, amount)** débite la source et crédite la destination du même montant ; en cas d'échec, aucun solde ne change.

AC-06 **SavingAccount.addPeriodicInterest()** ajoute *solde x interestRate* au solde.

AC-07 **CheckingAccount** autorise *FREE_TRANSACTIONS* opérations gratuites par mois ; **deductFees()** prélève *TRANSACTION_FEE* pour chaque opération au-delà du quota, puis remet le compteur à zéro.

AC-08 **Bank.search(num)** retourne le compte s'il existe, sinon un message d'erreur.

AC-09 **Bank.create(account)** ajoute un nouveau compte ; ajouter un compte déjà enregistré produit une erreur.

AC-10 **Bank.list()** retourne tous les comptes triés par solde.

AC-11 **Bank.delete(account)** supprime le compte ; supprimer un compte inconnu produit une erreur.

AC-12 **Bank.receipt(account)** affiche l'en-tête (*Account : numéro date :*), la ligne des colonnes (*date/time operation amount*) et une ligne par opération.

AC-13 **Menu** affiche le titre et les 9 options ; **choice()** retourne le numéro saisi ; une saisie invalide est redemandée.

AC-14 **App** boucle sur le menu jusqu'à l'option 9 (Exit) et ne plante jamais sur une saisie invalide ou une erreur métier.

2. Cas de test

2.1 Comptes (BankAccount, SavingAccount, CheckingAccount)

ID	CA	Scénario	Étapes	Résultat attendu
TC-01	AC-01	Constructeur par défaut	`new BankAccount()`	solde = 0,00
TC-02	AC-01	Constructeur avec montant initial	`new BankAccount(500)`	solde = 500,00
TC-03	AC-01	Montant initial négatif	`new BankAccount(-10)`	erreur, aucun compte créé
TC-04	AC-02	Auto-incrément	créer 3 comptes	les ids sont n, n+1, n+2
TC-05	AC-03	Dépôt valide	solde 100, `deposit(50)`	solde = 150,00

ID	CA	Scénario	Étapes	Résultat attendu
TC-06	AC-03	Dépôt nul ou négatif	`deposit(0)`, `deposit(-5)`	erreur, solde inchangé
TC-07	AC-04	Retrait valide	solde 100, `withdraw(40)`	solde = 60,00
TC-08	AC-04	Retrait du solde exact	solde 100, `withdraw(100)`	solde = 0,00
TC-09	AC-04	Solde insuffisant	solde 100, `withdraw(150)`	erreur, solde = 100,00
TC-10	AC-05	Virement valide	A=200, B=50, `A.transfer(B,80)`	A=120, B=130
TC-11	AC-05	Virement avec solde insuffisant	A=50, `A.transfer(B,80)`	erreur, A et B inchangés
TC-12	AC-05	Virement vers soi-même	`A.transfer(A,10)`	erreur, solde inchangé
TC-13	AC-06	Intérêts	SavingAccount 1000, taux 0,05, `addPeriodicInterest()`	solde = 1050,00
TC-14	AC-06	Intérêts sur compte vide	SavingAccount 0, taux 0,05	solde = 0,00
TC-15	AC-07	Dans le quota gratuit	Checking, 3 opérations, `deductFees()`	aucun frais
TC-16	AC-07	Au-delà du quota gratuit	Checking 100, 5 opérations, `deductFees()`	solde diminué de 2 x 0,50 = 1,00
TC-17	AC-07	Remise à zéro du compteur	après `deductFees()`	transactionCount = 0

2.2 Bank

ID	CA	Scénario	Étapes	Résultat attendu
TC-18	AC-08	Recherche d'un compte existant	`search(1)` après création du compte 1	retourne le compte 1
TC-19	AC-08	Recherche d'un compte inconnu	`search(99)`	message d'erreur : le compte 99 n'existe pas
TC-20	AC-09	Création	`create(account)`	le compte est présent dans la banque
TC-21	AC-09	Création d'un doublon	`create` du même compte deux fois	erreur au second appel, un seul compte stocké
TC-22	AC-10	Liste triée	soldes 300, 100, 200	ordre de la liste : 100, 200, 300
TC-23	AC-10	Liste vide	aucun compte	résultat vide, pas de plantage
TC-24	AC-11	Suppression d'un compte existant	`delete(account)`	`search` sur son id échoue désormais
TC-25	AC-11	Suppression d'un compte inconnu	`delete` d'un compte jamais créé	erreur
TC-26	AC-12	Contenu du relevé	dépôt 150, retrait 100, `receipt`	en-tête avec numéro de compte et date, ligne des colonnes, une ligne par opération (date/heure, opération, montant)
TC-27	AC-12	Relevé d'un compte vide	`receipt`	en-tête et ligne des colonnes uniquement

2.3 Menu et App (de bout en bout)

ID	CA	Scénario	Saisies	Résultat attendu
TC-28	AC-13	Affichage du menu	lancer l'application	titre *Account Management*, options 1 à 9, invite *Tapez votre choix :*
TC-29	AC-13	Choix invalide	saisir `0`, `12`, `abc`	l'invite est répétée, pas de plantage
TC-30	AC-14	Création d'un compte épargne	`3`, `1`, `1000`, `0.05`	message de création (SavingAccount, solde 1000,00)
TC-31	AC-14	Création d'un compte courant	`3`, `2`, `200`	message de création (CheckingAccount, solde 200,00)
TC-32	AC-14	Dépôt puis retrait	`5`, id, `150`, puis `6`, id, `100`	nouveaux soldes affichés après chaque opération
TC-33	AC-14	Retrait trop élevé	`6`, id, montant supérieur au solde	message d'erreur, menu réaffiché
TC-34	AC-14	Virement	`7`, source, destination, montant	*Virement effectue.*, les deux soldes sont mis à jour
TC-35	AC-14	Compte inconnu	`2`, `999`	message d'erreur, menu réaffiché
TC-36	AC-14	Montant non numérique	`5`, id, `abc`	redemande un nombre valide
TC-37	AC-14	Lister tous les comptes	`1` avec plusieurs comptes	comptes affichés triés par solde
TC-38	AC-14	Suppression	`4`, id	compte supprimé ; `1` ne l'affiche plus
TC-39	AC-14	Sortie	`9`	le programme se termine proprement

3. Définition de « terminé »

• Les 14 critères d'acceptation sont validés.
• Les 39 cas de test passent sans exception non gérée.
• Le code compile avec `javac *.java` et se lance avec `java App`.